



PostgreSQL

WHAT IS POSTGRESQL?

PostgreSQL is a powerful, open-source relational database management system (RDBMS). It is designed to store, manage, and query structured data using SQL (Structured Query Language).

It is widely used for:

- Web applications
- Data analytics
- Financial systems
- Enterprise backend systems



BRIEF HISTORY

Origins (1980s)

PostgreSQL traces its roots back to the University of California, Berkeley.

The project began in 1986 under the leadership of Faculty Member Michael Stonebraker, who had previously worked on the influential relational database system Ingres.

After Ingres, he launched a new research project called POSTGRES (Post-Ingres), aiming to address limitations of traditional relational databases.

BRIEF HISTORY

The POSTGRES project (1986–1994)

POSTGRES introduced several advanced concepts that were innovative at the time:

- User-defined data types
- Extensible operators
- Rules and triggers
- Object-relational features
- Support for complex data structures

The first versions were developed primarily for research and academic use.

BRIEF HISTORY

Transition to PostgreSQL (1994–1996)

In 1994, two Berkeley graduate students (Andrew Yu and Jolly Chen) added support for the SQL language to POSTGRES.

The new version was named Postgres95.

In 1996, the project was renamed **PostgreSQL** to reflect its SQL support while preserving its POSTGRES heritage.

BRIEF HISTORY

Open-source development (1996–present)

Since 1996, PostgreSQL has been maintained by a global open-source community coordinated through the PostgreSQL Global Development Group.



Unlike some database systems, PostgreSQL is not owned by a single company. Development is carried out collaboratively by contributors worldwide.

Official project website: [PostgreSQL Official Website](https://www.postgresql.org/)

BRIEF HISTORY

Major milestones

Year	Event
1986	POSTGRES project starts at UC Berkeley
1989	First academic release
1994	SQL support added (Postgres95)
1996	Renamed PostgreSQL
2005	Native tablespaces introduced
2010	Streaming replication added
2016	Parallel query execution introduced
2022	PostgreSQL 15 released
2025	PostgreSQL 18 released

STORAGE MODELS (RDBMS & NOSQL VS DISTRIBUTED FILE SYSTEMS)

OLTP ↔ NoSQL

NoSQL databases address OLTP (Online Transaction Processing) requirements in Big Data environments:

- Fast random access to data
- Frequent updates and writes
- Low latency (typically less than 10 ms)
- Support for multiple concurrent users
- Real-time application logic and processing

Why NoSQL Is Well-Suited for OLTP Workloads

- Key-value and column-oriented architectures provide very fast data lookups.
- Partial updates can be performed efficiently without rewriting entire records.
- Automatic horizontal scalability.
- High resilience and availability.

Typical use cases :

- User profiles
- Real-time features for online machine learning models
- Indexed logs and event data
- Web and mobile applications
- Transaction history with fast write operations

STORAGE MODELS (RDBMS & NOSQL VS DISTRIBUTED FILE SYSTEMS)

In a few short sentences

OLTP = operational service = RDBMS & NoSQL (PostgreSQL, MongoDB, Cassandra, HBase, Redis...)

**RDBMS & NoSQL =
OLTP = fast and frequent
operations**

“RDBMS and NoSQL are used to power applications running in production.”

“RDBMS and NoSQL are designed for production use.”

V/S

V/S

V/S

V/S

OLAP = data lake = HDFS/S3 + analytical engines (Hive, Spark, Presto)

**Distributed file system =
OLAP = big data analytics**

“Distributed file systems are used to store and analyze raw data at large scale.”

“Distributed file systems are designed for data analysis.”

CORE CONCEPTS

Relational Database

Data is stored in:

- Tables (rows + columns)
- Relationships between tables (via keys)

Example:

- users table
- orders table
- Each order belongs to a user via a foreign key

CORE CONCEPTS

SQL support

PostgreSQL uses SQL to interact with data:

```
SELECT * FROM users WHERE id = 1;
```

You can:

- insert data
- query data
- update data
- delete data

CORE CONCEPTS

ACID compliance

PostgreSQL ensures reliable transactions using ACID:

- Atomicity (all or nothing)
- Consistency (valid data rules)
- Isolation (transactions don't interfere)
- Durability (data survives crashes)

This makes it very reliable for critical systems.

<https://www.scylladb.com/>



CORE CONCEPTS

Some advanced features

PostgreSQL is more advanced than many basic SQL databases:

- JSON / JSONB support (semi-structured data)
- Full-text search
- Indexing (B-tree, GIN, GiST)
- Stored procedures (functions)
- Triggers
- Extensions (e.g. PostGIS for geospatial data)

CORE CONCEPTS

Architecture basics

- PostgreSQL runs as a server process
- It accepts connections from clients (apps, tools, etc.)

Key components:

- Database: container of data
- Schema: logical grouping inside a database
- Table: stores rows of data

CORE CONCEPTS

Default settings (important for Docker)

When using PostgreSQL in Docker:

- Default port: 5432
- Default user: often postgres
- Data directory: /var/lib/postgresql/data

Environment variables often used:

- POSTGRES_PASSWORD
- POSTGRES_USER
- POSTGRES_DB

CORE CONCEPTS

A basic example

Create a table:

```
CREATE TABLE users (  
    id SERIAL PRIMARY KEY,  
    name TEXT NOT NULL,  
    email TEXT UNIQUE  
);
```

Insert data:

```
INSERT INTO users (name, email)  
VALUES ('Alice', 'alice@example.com');
```

Query data:

```
SELECT * FROM users;
```


CORE CONCEPTS

Why is PostgreSQL so popular?

- Free and open-source
- Very stable and production-ready
- Highly extensible
- Strong consistency and reliability
- Used by startups and large enterprises



POSTGRESQL AND DOCKER

Docker allows you to:

- run reproducible environments
- manage vPostgreSQL without installing it locally
- isolate the database in a container
- quickly create versions easily (e.g., PostgreSQL 15, 16, etc.)

A typical setup uses Docker Compose to define:

- a PostgreSQL service
- persistent storage (volumes)
- environment variables (database, user, password)

This is widely used in development and teaching because it makes database setup fast, clean, and portable.

POSTGRESQL AND DOCKER

The official Docker image is postgres (maintained on Docker Hub).

https://hub.docker.com/_/postgres

CORE CONCEPTS

Also remember the default settings (they are not specific to Docker as they come from PostgreSQL itself, but they are important for Docker)

When using PostgreSQL in Docker:

- Default port: 5432
- Default user: often postgres
- Data directory: /var/lib/postgresql/data

Environment variables often used:

- POSTGRES_PASSWORD
- POSTGRES_USER
- POSTGRES_DB



← Back to blog

PostgreSQL Query: Introduction, Explanation, and 50 Examples

<https://www.enterprisedb.com/postgres-tutorials/postgresql-query-introduction-explanation-and-50-examples>

LAB

PYTHON + DOCKER + POSTGRESQL

INTEGRATION WITH PYTHON

To connect PostgreSQL running in Docker from Python, you typically use a PostgreSQL driver such as **psycopg**.

Install a driver:

```
pip install psycopg2-binary
```

(or modern alternative psycopg v3)

INTEGRATION WITH PYTHON



Let's have a look at our postgresql example script.

```
import psycopg2

conn = psycopg2.connect(
    host="db",
    database="app_db",
    user="admin",
    password="admin123",
    port=5432
)

cur = conn.cursor()
cur.execute("SELECT version();")
print(cur.fetchone())

cur.close()
conn.close()
```


INTEGRATION WITH PYTHON

SQLAlchemy

SQLAlchemy is a Python library that lets you interact with databases like PostgreSQL using Python objects instead of writing raw SQL everywhere.

When combined with Docker, it's a very common stack for modern backend development.

You can use it along with psycopg.

```
pip install sqlalchemy psycopg2-binary
```

INTEGRATION WITH PYTHON

Psycopg:

- Low-level driver for PostgreSQL
- You write SQL manually

SQLAlchemy:

- Higher-level toolkit + ORM
- Can generate SQL or abstract it away



Simplicity? Discussion

WHAT IS THE OFFICIAL DOCKER IMAGE USED TO RUN POSTGRESQL?

- ☐ MYSQL
- ☐ POSTGRES
- ☐ POSTGRESQL
- ☐ POSTGRESQL-DB
- ☐ DB-POSTGRE

WHAT IS THE OFFICIAL DOCKER IMAGE USED TO RUN POSTGRESQL?

- ☐ MYSQL
- ☒ POSTGRES
- ☐ POSTGRESQL
- ☐ POSTGRESQL-DB
- ☐ DB-POSTGRE

IN A DOCKER COMPOSE SETUP,
WHAT SHOULD A PYTHON
APPLICATION USE AS THE
DATABASE HOST?

- ☐ LOCALHOST
- ☐ 127.0.0.1
- ☐ THE CONTAINER IP ADDRESS
- ☐ THE SERVICE NAME (EX. "DB")

IN A DOCKER COMPOSE SETUP,
WHAT SHOULD A PYTHON
APPLICATION USE AS THE
DATABASE HOST?

☐ LOCALHOST

☐ 127.0.0.1

☐ THE CONTAINER IP ADDRESS

☒ THE SERVICE NAME (EX. "DB")

WHICH PYTHON LIBRARY IS
COMMONLY USED TO CONNECT
DIRECTLY TO POSTGRESQL
WITHOUT AN ORM?

- ☐ PYMONGO
- ☐ PSYCOPG2
- ☐ SQLITE3
- ☐ SQLALCHEMY

WHICH PYTHON LIBRARY IS
COMMONLY USED TO CONNECT
DIRECTLY TO POSTGRESQL
WITHOUT AN ORM?

☐ PYMONGO

☒ PSYCOPG2

☐ SQLITE3

☐ SQLALCHEMY

WHAT IS THE MAIN ADVANTAGE OF SQLALCHEMY OVER RAW PSYCOPG?

- ❑ IT IS A HIGH-PERFORMANCE REPLACEMENT FOR POSTGRESQL
- ❑ IT AUTOMATICALLY INSTALLS DOCKER
- ❑ IT PROVIDES AN ORM AND ABSTRACTION OVER SQL
- ❑ IT WORKS WITH JSON DATABASES

WHAT IS THE MAIN ADVANTAGE OF SQLALCHEMY OVER RAW PSYCOPG?

- ☐ IT IS A HIGH-PERFORMANCE REPLACEMENT FOR POSTGRESQL
- ☐ IT AUTOMATICALLY INSTALLS DOCKER
- ☒ IT PROVIDES AN ORM AND ABSTRACTION OVER SQL
- ☐ IT WORKS WITH JSON DATABASES



Once again, create three to five questions about PostgreSQL and its integration with Docker and/or Python.

**Multiple choice questions
4 options, 1 correct answer**